

Markdown 技术文档基础译前处理指引

Contents

Chapter 1. 简介	4
Chapter 2. 适用范围与边界	5
Chapter 3. 开始前准备	6
工具准备	6
文件状态与目标确认	6
处理前基本原则	6
Chapter 4. 需要重点识别的内容类型	7
Markdown 标记	7
变量与占位符	7
代码块与命令行内容	7
链接与路径	8
源文格式异常	8
Chapter 5. 基础处理流程	10
初步扫描文件结构	10
使用 Regex 定位高风险内容	11
标记或保护不可直接处理的内容	12
清理源文格式异常	13
处理后复核	14
Chapter 6. 常见 Regex 应用场景	16
识别变量与占位符	16
识别 Markdown 链接	16
识别源文格式异常	17
Chapter 7. 参考示例：Markdown 待译源文的基础译前处理	18
输入片段	18
识别结果与处理判断	19
处理过程示意	19
处理后结果	20

输出物与复核结论.....	21
Chapter 8. 常见问题与风险.....	22
Chapter 9. 交付前检查清单.....	23
交付物.....	23
文件与版本确认.....	23
基础扫描与定位.....	23
标记、保护与清理.....	23
复核与交付准备.....	23
Chapter 10. 总结.....	24

Chapter 1. 简介

本文用于指导 Markdown 待译技术文档在导入 CAT 工具前完成基础风险识别、必要标记、有限清理和结果复核。

面向读者：

- 需要对 Markdown 待译源文进行基础预处理的本地化执行人员
- 需要复核处理结果的项目协调人员或审校人员

目标：

- 在导入 CAT 工具前完成 Markdown 源文的基础风险识别
- 标记或保护不可直接处理的内容
- 清理少量规则明确的格式异常
- 输出可复核、可回退的处理结果和检查记录

本文仅覆盖 Markdown 技术文档基础译前处理中的常见识别与初步处理任务，不涉及复杂脚本自动化、结构化 XML / DITA 文档处理或完整 CAT 工作流设计。

Chapter 2. 适用范围与边界

本文适用于安装说明、操作指南、产品帮助文档、开发者说明和知识库页面等 Markdown 源文件的基础译前处理。

适用任务：

- 识别高风险内容
- 检查源文是否存在明显格式问题
- 定位变量、占位符、链接、代码相关内容
- 对少量规则明确的格式异常做有限清理
- 形成可复核的初步处理结果

不适用任务：

- 复杂嵌套规则
- 跨文件工程级处理
- 以 Regex 学习为主的培训场景
- 深度源文重构场景

Chapter 3. 开始前准备

在正式进行译前处理之前，应先确认工具、文件状态和处理边界。若缺少基本确认，后续处理中很容易误改原文或遗漏高风险内容。

工具准备

最低工具要求：

必需：

- 可直接查看和编辑 Markdown 源文的文本编辑器
- 支持 Regex 查找
- 可保留处理前版本的备份方式

执行前请确认编辑器已启用 Regex 查找模式。

文件状态与目标确认

检查项	确认要求
文件格式	为可编辑 Markdown 源文件
版本状态	为当前最终待译版本
处理范围	本次仅做基础译前处理
历史痕迹	已确认是否有旧标记/批注残留
记录要求	已确认是否需保留处理痕迹

处理前基本原则

- 先识别，再处理
- 以源文安全为优先
- 小范围试验，再批量操作
- 保留原文件或可回退版本
- 处理动作应可解释、可复核
- Regex 只用于辅助识别和基础处理

Chapter 4. 需要重点识别的内容类型

在 Markdown 技术文档的译前处理中，应优先识别那些结构相对固定、出现频率较高，且一旦误改就可能影响翻译、导入或显示结果的内容。

本节列出需要重点识别的内容类型，并说明其主要风险、关注点和基础处理方式。

Markdown 标记

典型示例：

- `# 安装说明`
- `**注意**`
- ``npm install``

风险： 误删或改坏结构边界可能导致格式错乱或渲染异常。

关注点： 区分结构标记与普通正文，确认边界是否完整。

如何处理： 优先定位，不作为普通正文直接处理。

变量与占位符

典型示例：

- `{name}`
- `{{username}}`
- `%s、%d`

风险： 误改后可能导致变量替换失败或显示异常。

关注点： 确认写法是否固定，是否存在多种格式并存。

如何处理： 优先定位，保留原样；必要时列入保护或人工确认清单。

代码块与命令行内容

典型示例：

- 代码块示例：

```
npm install
```

- 行内命令：`cd /app/project`
- 带参数的命令：`python app.py --port 8080`

风险：字符、空格和参数常不可自由改写，误改会导致示例失效。

关注点：确认代码块边界和命令语境。

如何处理：不作为普通正文直接处理。

链接与路径

典型示例：

- Markdown 链接：`[安装说明](https://example.com/install)`
- 文件路径：`/usr/local/bin`
- 相对路径：`./images/logo.png`

风险：误改链接结构、地址内容或路径写法，可能导致跳转失效、资源找不到，或示例无法正常使用。

关注点：区分链接文本与实际地址，确认路径是否属于代码、命令或其他结构敏感内容的一部分。

如何处理：优先定位，保留链接地址与路径原样；如边界异常或写法不完整，列入人工确认清单。

源文格式异常

典型示例：

- 多余空格：`Click the button`
- 异常换行：

```
Please install  
the package first.
```

- 重复空行：

```
## 安装
```

```
运行以下命令。
```


风险：明显的格式异常可能影响阅读、查找、一致性以及后续导入处理；若在未确认上下文的情况下直接批量清理，也可能破坏原有结构。

关注点：先判断是否确实属于异常格式，而不是有意保留的排版形式；尤其注意代码块、列表、链接和命令附近的空格与换行。

如何处理：仅对规则明确、风险较低的问题进行有限清理；边界不清时先保留原状，并列入人工确认清单。

Chapter 5. 基础处理流程

在完成处理对象识别后，下一步进入基础处理流程。本节说明如何按较稳妥的顺序，对已识别的高风险内容进行扫描、定位、标记、有限清理和复核。

以下步骤按执行顺序展开。每一步包含目标、具体动作、输出结果、验收标准和注意事项，便于实际操作和后续复核。

在执行以下步骤时，建议同步保留处理记录，以便后续复核、交接或回退。

处理记录至少应包含以下内容：

- 当前处理文件的版本信息
- 已执行的查找对象类别或 Regex 规则
- 已完成的标记、保护或清理动作
- 暂未处理并保留原状的内容类型
- 需人工确认的内容及其位置

初步扫描文件结构

目标

从整体上了解 Markdown 技术文档的基本结构，判断正文、标题、列表、代码块、命令、链接与路径等内容的大致分布，为后续处理建立基础。

具体动作

1. 用支持纯文本查看的编辑器打开 Markdown 技术文档，优先查看源文，而不是渲染后的页面。
2. 快速浏览全文，确认标题层级、列表结构、代码块与命令行内容、链接与路径等常见结构的大致分布。
3. 额外留意高风险内容较集中的区域，例如连续代码块、命令示例区、参考链接区或异常空行较多的部分。
4. 如有需要，可先做简单标记或记录，注明后续需要重点检查的区域，但此时不进行批量替换或大范围修改。

输出结果

- 哪些区域以普通正文为主
- 哪些区域包含较集中的高风险内容
- 哪些位置存在需重点检查的格式异常

验收标准

- 已确认文件为可直接处理的 Markdown 源文，而非仅可查看的渲染页面
- 已完成对标题、列表、代码块、命令示例、链接与路径分布的整体浏览
- 已识别高风险内容较集中的区域
- 在完成初步扫描前，未进行批量替换或大范围修改

注意事项

- 不要在全面扫描前进行任何局部修改
- 不要在初步扫描阶段直接进入批量查找与替换
- 若发现文档结构混乱、版本不清或内容仍在频繁变动，应先暂停处理

使用 Regex 定位高风险内容

目标

在完成整体扫描后，使用 Regex 对重点对象进行规则化定位。

具体动作

1. 按类别分开执行查找，不要将 Markdown 标记、变量与占位符、代码块与命令行内容、链接与路径、源文格式异常混在一次查找中处理。
2. 先从结构较固定、风险较明确的内容开始，例如变量与占位符、Markdown 链接和简单格式标记。
3. 每次只使用一个目的明确的 Regex，先检查匹配结果是否符合预期，再决定是否继续扩大范围。
4. 对匹配结果进行抽查，并按类型区分为需保护内容、需人工确认内容或疑似格式异常内容。

输出结果

完成这一步后，通常应能得到以下结果：

- 已定位的变量与占位符位置
- 已定位的 Markdown 标记或结构片段
- 已定位的链接与路径
- 已初步识别的源文格式异常位置
- 一份按类型区分的后续处理清单

验收标准

- 已按类型分开执行查找，未将不同类别对象混在同一轮处理中
- 每条 Regex 的匹配结果均已完成基础抽查
- 已区分出需保护内容、需人工确认内容和疑似格式异常内容
- 对误匹配较多的规则，已缩小范围、调整规则或停止继续使用

注意事项

- 匹配结果只代表疑似符合某种结构，不代表这些内容一定可以直接处理
- 不要因为某条 Regex 能匹配到很多结果，就直接执行批量替换
- 对误匹配较多的规则，应先缩小范围或调整思路，而不是继续扩大使用
- 若某类内容边界不清，较稳妥的做法是先定位并保留，留待后续人工判断

标记或保护不可直接处理的内容

目标

对已定位的高风险内容进行初步区分，明确哪些内容应保留原样，哪些内容需标记保护，哪些内容需人工确认。

具体动作

1. 根据前一步的定位结果，优先处理变量与占位符、代码块与命令行内容、链接地址与路径等不可随意改写的内容。
2. 对用途明确、结构固定的高风险内容，可采用统一方式进行标记或保护，例如加入备注、使用内部处理标识，或在后续导入前列入单独检查清单。
3. 对边界不清、用途暂时无法确认的内容，先保留原状，并标记为“需人工确认”，不要直接替换或删除。
4. 如需进行临时保护，应确保保护方式不会破坏 Markdown 技术文档原有结构，也不会影响后续还原。
5. 对已标记或已保护的内容做一次快速回看，确认没有将普通正文误归入不可直接处理内容。

输出结果

完成这一步后，通常应能得到以下结果：

- 已被标记为不可直接处理的高风险内容
- 已被保护、暂不进入普通翻译处理的内容
- 已单独记录并待人工确认的可疑片段
- 一份更适合继续进行基础清理和后续复核的文件版本

验收标准

- 变量、占位符、代码块、命令、链接地址与路径等高风险内容已完成初步区分
- 需保留原样的内容未被当作普通正文直接修改

- 边界不清或用途暂不明确的内容已列入人工确认范围
- 所采用的标记或保护方式未破坏 Markdown 原有结构，且后续可回退或复核

注意事项

- 标记或保护的目的在于防止误改，而不是将所有特殊内容一次处理完毕
- 保护方式应尽量简单、可回退、可复核，不要引入新的复杂标记
- 不要为了追求统一，将用途不同的内容用同一种方式强行处理
- 若保护动作本身可能影响原文结构，应先缩小范围验证，再决定是否继续
- 边界不清时，应优先保留原状，避免提前改动高风险内容

清理源文格式异常

目标

在高风险内容完成初步标记或保护后，对规则明确、风险较低的源文格式异常进行有限清理。

具体动作

1. 优先处理边界清楚、可稳定判断的源文格式异常，例如明显多余的空格、重复空行，或不符合当前上下文的异常换行。
2. 使用 Regex 或普通查找逐类检查，不同类型的问题分开处理。
3. 每次处理前先抽查匹配结果，确认其确属格式异常，而非有意排版。
4. 小范围试改后立即查看结果，确认标题、列表、代码块与命令行内容、链接等区域未受到影响，再决定是否继续扩大处理范围。
5. 对无法稳定判断的情况，先保留原状，并记录为后续人工复核项。

输出结果

完成这一步后，通常应能得到以下结果：

- 已清理的明显多余空格
- 已整理的重复空行
- 已修正的部分异常换行
- 一份结构更稳定、但仍保留必要原始信息的 Markdown 技术文档

验收标准

- 已清理的问题均属于规则明确、边界清楚的格式异常
- 批量清理前已完成小范围试改和结果抽查

- 标题、列表、代码块、命令、链接和路径区域未因清理动作受到破坏
- 边界不清或无法稳定判断的内容仍保留原状，并已列入人工复核范围

注意事项

- 不要将“看起来不整齐”直接视为需要清理的格式异常
- 不要在未确认上下文的情况下批量合并空行或删除换行，尤其要避开代码块与命令行内容附近的区域
- 对列表、链接、路径及其他结构敏感区域中的空格处理要格外谨慎，避免误改有意义的内容
- 清理动作应以最小改动为原则，只处理规则明确、复核成本较低的问题

处理后复核

目标

在完成定位、标记或保护以及基础清理后，对处理结果进行一次集中复核。

具体动作

1. 重新浏览全文，重点检查标题、列表、代码块与命令行内容、链接与路径周边区域，确认结构未被破坏。
2. 对前面使用过的 Regex 再执行一次基础查找，确认关键高风险内容是否仍位于合理位置，或是否出现误删、误改、漏标记等情况。
3. 抽查已清理的源文格式异常，确认处理结果符合预期，没有将原本有意义的排版一并改动。
4. 对“需人工确认”的内容单独回看，确认是否仍需保留备注，或是否可以在当前阶段补充说明。
5. 保存处理后版本，并确保原文件或可回退版本仍可访问，便于后续翻译、审校或交接时追溯。

输出结果

完成这一步后，通常应能得到以下结果：

- 一份已完成基础译前处理的 Markdown 技术文档
- 一份结构仍然完整、可继续翻译或导入的文件版本
- 一组已确认保留、已处理或需人工确认的高风险内容记录
- 一份可回退的原文件或处理前版本

验收标准

- 处理后的 Markdown 文件结构完整，可继续用于翻译、导入或交接
- 关键高风险内容未出现误删、误改或漏标记

- 已清理的格式异常经抽查后符合预期
- 原文件或可回退版本仍可访问，且需人工确认项已单独保留记录

注意事项

- 复核不是重复前面所有动作，而是验证处理结果是否稳定、可继续使用
- 不要只检查被修改过的局部内容，也要留意是否连带影响到相邻区域
- 若复核中发现问题，应回到对应步骤进行局部修正，而不是在当前阶段继续扩大处理范围

Chapter 6. 常见 Regex 应用场景

本节列出几类在 Markdown 技术文档基础译前处理中较常见的 Regex 规则示例，用于辅助定位变量、链接和部分源文格式异常。

以下示例仅用于初步识别，不应脱离上下文直接作为批量替换依据。

识别变量与占位符

用途

用于初步识别结构固定的变量与占位符，便于后续标记、保护或人工确认。

示例

类型	原文示例	Regex 示例	用途说明
花括号变量	Hello, {name}!	\{[^{}]+\}	识别形如 {name} 的变量，便于先做基础定位
百分号占位符	File count: %d User: %s	%[a-zA-Z]	识别简单占位符，如 %d、%s，避免在后续处理中被误改

注意事项

- 不同项目中的变量与占位符写法可能不同，使用前应先抽查匹配结果，再决定是否继续扩大范围。

识别 Markdown 链接

用途

用于初步识别结构较标准的 Markdown 链接，便于检查链接边界是否完整。

示例

类型	原文示例	Regex 示例	用途说明
Markdown 链接	[安装说明] (https://example.com/install) [下载页面] (./docs/setup.md)	\[[^\[\]]+\]\([^\)]+\)	识别常见 Markdown 链接结构，便于先检查链接边界是否完整

注意事项

- 应区分链接文本和链接地址，不要将两部分混为一体处理。
- 对边界不完整或疑似写坏的链接，应优先标记并单独检查。

识别源文格式异常

用途

用于初步定位规则较明确的源文格式异常，便于后续做有限清理。

示例

原文示例：

```
Click the button to continue.
```

Regex 示例：

```
{2,}
```

原文示例：

```
Please install  
the package first.
```

Regex 示例：

```
\S\n\S
```

注意事项

- 列表、代码块、命令、链接和路径附近的空格与换行应谨慎处理。
- 若同一条 Regex 匹配结果过多，应先缩小范围并抽查结果。

Chapter 7. 参考示例：Markdown 待译源文的基础译前处理

本节通过一个简化示例，说明如何将前文所述的识别原则和基础处理流程应用到实际 Markdown 待译源文中。示例重点不在于覆盖所有情况，而在于展示一次完整的基础处理思路，包括识别、区分、有限清理和复核。

输入片段

以下为待处理的 Markdown 源文示例：

```
# 安装说明
```

```
请先使用 `{username}` 登录系统，然后访问[安装页面](https://example.com/install)。
```

```
配置文件路径为 `/usr/local/bin/app/config.yaml`。
```

```
如需切换目录，请运行 `cd /app/project`。
```

```
```bash
```

```
python app.py --port 8080
```

```
```
```

```
Please install
```

```
the package first.
```

```
Click the button to continue.
```

```
## 下一步
```

```
运行以下命令完成初始化。
```

该示例中同时包含以下内容：

- Markdown 标记
- 变量
- 链接
- 文件路径

- 行内命令
- 代码块与命令行内容
- 异常换行
- 多余空格
- 重复空行

识别结果与处理判断

根据前文流程，可先对示例中的高风险内容和格式异常进行初步识别与分类。处理重点不是一次性改动全部内容，而是先明确哪些内容应保留原样，哪些内容可以清理，哪些内容需要后续人工确认。

| 内容片段 | 类型 | 初步处理判断 | 原因 |
|---|-------------|---------------|-----------------------|
| # 安装说明 | Markdown 标记 | 保留原样 | 属于标题结构标记，直接影响层级与显示 |
| {username} | 变量 | 保留原样并列入检查范围 | 属于动态字段，不应作为普通正文改写 |
| [安装页面]
(https://example.com/install) | Markdown 链接 | 保留原样，检查结构是否完整 | 需同时保留链接文本与地址结构 |
| /usr/local/bin/app/config.yaml | 文件路径 | 保留原样 | 路径属于结构敏感内容，误改可能导致定位错误 |
| cd /app/project | 行内命令 | 保留原样 | 命令内容不可作为普通正文处理 |
| python app.py --port 8080 | 代码块命令 | 保留原样 | 参数、空格和写法可能影响执行结果 |
| Please install the package first. | 异常换行 | 列入格式异常检查 | 需判断是否属于非预期断行 |
| Click the button to continue. | 多余空格 | 可清理 | 属于规则明确的格式异常 |
| 标题前后的连续空行 | 重复空行 | 可清理 | 在不影响结构的前提下可做有限整理 |

处理过程示意

根据上述识别结果，可按以下顺序进行基础处理：

- 1. 先确认标题、链接、变量、路径、行内命令和代码块均属于结构敏感内容，不作为普通正文直接修改。
- 2. 对 {username}、链接地址、文件路径、行内命令和代码块命令保持原样，并在记录中注明已完成初步保护或保留。
- 3. 对

```
Please install  
the package first.
```

进行上下文判断。若确认其不属于有意换行，则可合并为正常句子。

- 4. 对 Click the button to continue. 中连续两个空格进行有限清理。
- 5. 对标题前后的重复空行进行整理，但不破坏标题层级与段落分隔。
- 6. 完成处理后重新检查 Markdown 结构，确认标题、链接、代码块和路径未受影响。

处理后结果

在仅处理规则明确的问题，并保留结构敏感内容原样的前提下，示例可整理为以下形式：

```
# 安装说明  
  
请先使用 {username} 登录系统，然后访问[安装页面](https://example.com/install)。  
  
配置文件路径为 /usr/local/bin/app/config.yaml。  
  
如需切换目录，请运行 cd /app/project。  
  
```bash  
python app.py --port 8080
```  
  
Please install the package first.  
  
Click the button to continue.  
  
## 下一步  
  
运行以下命令完成初始化。
```

输出物与复核结论

完成本示例处理后，通常应输出以下内容：

- 一份已完成基础整理的 Markdown 文件
- 一份处理记录，包括已保留原样的结构敏感内容、已执行的清理动作和需人工确认项
- 一份可回退的原文件或备份版本

复核时，应至少确认以下几点：

- Markdown 标题结构保持完整
- 变量、链接、路径、行内命令和代码块内容未被误改
- 已处理的空格、空行和换行问题均属于规则明确的格式异常
- 当前版本可继续进入后续翻译、导入或交接流程

Chapter 8. 常见问题与风险

在 Markdown 技术文档的译前处理中，Regex 可以帮助快速定位问题，但也可能因使用方式不当而引入新的风险。常见问题通常不在于“不会写 Regex”，而在于处理范围失控、误改结构敏感内容、误判格式异常，或在处理后缺少复核。

本节将常见问题整理为以下几类，便于在实际操作中快速对照检查。

| 问题类型 | 主要风险 | 常见原因 | 避免方式 |
|--------------------|--|--|---|
| 匹配范围失控 | 将不属于当前目标的内容一并匹配或替换，导致判断混乱，严重时可能误改正文或破坏结构 | Regex 写得过宽，或在未抽查结果前直接扩大使用范围 | 先小范围定位；每次查找后先抽查结果；误匹配较多时优先调整规则，而不是继续扩大范围 |
| 误改结构敏感内容 | 导致变量失效、链接损坏、路径错误、命令不可用或代码示例失效 | 将变量、链接、路径、代码块或命令当作普通正文处理，或未先确认边界即直接修改 | 先单独识别结构敏感内容；变量、链接地址、路径、命令和代码块通常保留原样；边界不清时先标记并人工确认 |
| 误清理有意排版 | 误删有意义的空行、换行或结构格式，影响 Markdown 显示和后续处理 | 仅根据外观判断异常，未结合上下文确认其是否承担结构作用 | 先确认是否确属格式异常；对标题、列表、代码块、命令和链接附近的空格与换行格外谨慎；只清理规则明确的问题 |
| 过度依赖 Regex 或处理后未复核 | 将匹配结果直接当作处理结论，或遗漏处理过程中引入的新问题，导致最终版本不稳定 | 误以为“能匹配到”就等于“能直接处理”，或过于关注处理动作本身而忽略结果验证 | 将匹配结果视为线索而非结论；复杂情况单独人工确认；处理后重新执行关键查找并抽查重点区域，确保结果可继续使用 |

这一节中的问题大多不是工具本身造成的，而是使用边界不清导致的。只要保持“小范围验证、分类处理、结果复核”的基本习惯，Regex 在 Markdown 技术文档译前处理中通常能发挥稳定的辅助作用。

Chapter 9. 交付前检查清单

交付物

- 已生成处理后的 Markdown 文件
- 已生成需人工确认项记录
- 已保留原文件或可回退版本

文件与版本确认

- 已确认当前文件为可继续使用的 Markdown 技术文档源文
- 已确认处理对象为当前最终待译版本
- 已确认处理范围仅限基础译前处理

基础扫描与定位

- 已完成对全文结构的基础扫描
- 已按类型定位 Markdown 标记
- 已按类型定位变量与占位符
- 已按类型定位代码块与命令行内容
- 已按类型定位链接与路径
- 已识别需处理的源文格式异常

标记、保护与清理

- 不可直接处理的内容已完成标记或保护
- 未将高风险内容当作普通正文直接修改
- 源文格式异常仅在规则明确时进行了基础清理
- 未对边界不清的内容执行批量替换
- 暂未处理的内容已保留原状，并列入人工确认范围

复核与交付准备

- 已对处理结果进行复核
- 已抽查标题、列表、代码块与命令行内容、链接与路径周边区域
- 已确认关键结构未被破坏
- 已确认处理后文件可继续用于翻译、导入或交接
- 已保留处理记录，便于后续追溯

Chapter 10. 总结

本文提供 Markdown 待译源文的基础译前处理指引，用于在导入 CAT 工具前完成风险识别、必要标记、有限清理和结果复核。

在实际处理过程中，更稳妥的做法始终是先识别对象，再按类型处理，并在处理后完成复核。Regex 在这一过程中主要用于辅助定位和初步识别，而不替代人工判断。

本文的目标不是一次性解决所有复杂问题，而是输出一份结构稳定、风险可见、可继续进入后续翻译流程的文件。